If the actor receives a positive response from the market, it sends itself `MarketOddsAvailable(true,[odds])` to indicate that the validation has passed, along with the odds; otherwise, it sends `MarketOddsAvailable(false, [odds])`. Sending the odds helps operators understand the system's behavior by registering what odds were requested for the bet.

Listing 12.11 Market validation request

```
object Bet {
  ...

  private def requestMarketStatus(
      command: Open,
      sharding: ClusterSharding,
      context: ActorContext[Command]): Unit = {
    val marketRef =
      sharding.entityRefFor(Market.typeKey, command.marketId)    ❶

    implicit val timeout: Timeout = Timeout(3, SECONDS)          ❷
    context.ask(marketRef, Market.GetState) {                    ❸
      case Success(Market.CurrentState(marketState)) =>
        val matched = oddsDoMatch(marketState, command)          ❹
        MarketOddsAvailable(
          matched.doMatch,
          Option(matched.marketOdds))
      case Failure(ex) =>                                        ❺
        context.log.error(ex.getMessage())
        MarketOddsAvailable(false, None)
    }
  }
  ...
}
```

❶ Finds an ActorRef for the market

❷ Sets the timeout for .ask to 3 seconds

❸ asks the market its state

❹ Verifies that the market's odds match the bet's odds

❺ A failure marks the odds as unavailable.